

Relacion-Ejercicios-Tema-4.pdf



CodeWolf



Configuración y Evaluación de Sistemas Informáticos



3º Grado en Ingeniería Informática



**Escuela Politécnica Superior de Córdoba
Universidad de Córdoba**

RELACIÓN 4 CESI



WUOLAH

PROBLEMA 4.1 En un sistema Linux se ha ejecutado la orden `uptime` tres veces en momentos diferentes. El resultado, de forma resumida, es el siguiente:

```
... load average: 6.85, 7.37, 7.83
... load average: 8.50, 10.93, 8.61
... load average: 37.34, 9.47, 3.30
```

Indique si la carga crece, decrece, se mantiene estacionaria o bien no puede decidir sobre ello.

No se puede decidir ya que no sabemos si las tres ejecuciones pertenecen a un mismo espacio de tiempo próximo.

PROBLEMA 4.2 En un sistema Linux se ha ejecutado la siguiente orden:

```
$ time quicksort
real 0m40.2s
user 0m17.1s
sys 0m3.2s
```

Indique si el sistema está soportando mucha o poca carga. Razone la respuesta.

Sabemos que el comando `time` muestra el tiempo que tarda un proceso en ejecutarse; por lo cual el programa `quicksort` ha tardado 40'2 s pero también sabemos que el tiempo necesario por el programa es la suma del tiempo `user` y `sys`, dicha suma daría 20'3 s. Comparando estos tiempos podemos decir que el sistema está soportando mucha carga ya que ha necesitado casi el doble de tiempo.

PROBLEMA 4.3 Considere las órdenes siguientes ejecutadas en un sistema Linux:

```
$ time simulador_original  
real 0m24.2s  
user 0m15.1s  
sys 0m1.6s
```

```
$ time simulador_mejorado  
real 0m32.8s  
user 0m10.7s  
sys 0m2.1s
```

1. ¿Cuál es el tiempo de ejecución de ambos simuladores?
2. Calcule, si es el caso, la mejora en el tiempo de ejecución del simulador mejorado respecto del original.

① Programa Simulador-original | Programa simulador-mejorado
 $T = 24'2s$; tiempo necesario: $16'7s$ | $T = 32'8s$ tiempo necesario: $12'8s$

② $G = \frac{t_o}{t_m} = \frac{16'7}{12'8} = 1'30468$

PROBLEMA 4.4 Se sabe que la sobrecarga (overhead) de un monitor software sobre un computador es del 4 %. Si el monitor se activa cada 2 segundos, ¿cuánto tiempo tarda el monitor en ejecutarse por cada activación?

Sobrecarga = 4% = 0'04

Carga Sistema = 2 s

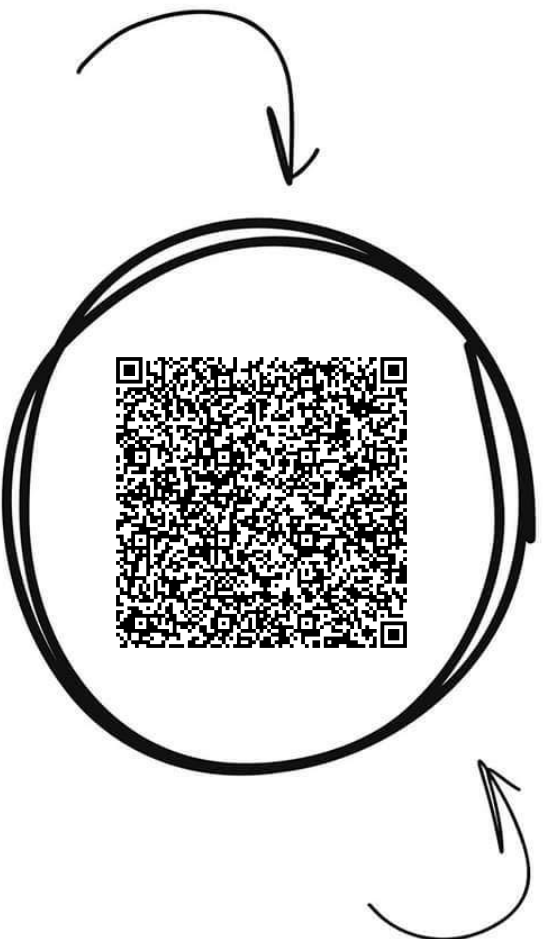
Debemos de utilizar la formula $\text{Sobrecarga} = \frac{\text{Carga Monitor}}{\text{Carga Sistema}}$

$\boxed{\text{Carga Monitor} = 0'04 \cdot 2 = 0'08s}$

Configuración y Evaluación d...



Comparte estos flyers en tu clase y consigue más dinero y recompensas



Banco de apuntes de la

WUOLAH

- 1** Imprime esta hoja
- 2** Recorta por la mitad
- 3** Coloca en un lugar visible para que tus compis puedan escanar y acceder a apuntes

- 4** Llévate dinero por cada descarga de los documentos descargados a través de tu QR



PROBLEMA 4.5 El monitor sar (system activity reporter) de un computador se activa cada 15 minutos y tarda 750 ms en ejecutarse por cada activación. Se pide:

- Calcular la sobrecarga que genera este monitor sobre el sistema informático.
- Si la información generada en cada activación ocupa 8192 bytes, ¿cuántos ficheros históricos del tipo saDD se pueden almacenar en el directorio /var/log/sa si se dispone únicamente de 200 MB de capacidad libre?

$$\text{Carga Sistema} = 15 \text{ min} = 15 \cdot 60 \text{ s}$$

$$\text{Carga Monitor} = 750 \text{ ms} = 0.75 \text{ s}$$

$$\text{Sobrecarga} = \frac{0.75}{15 \cdot 60} = 8.3 \cdot 10^{-4}$$

Cada activación ocupa 8192 bytes

$$\frac{8192 \text{ bytes}}{15 \text{ min}} \cdot \frac{60 \text{ min}}{1 \text{ h}} \cdot \frac{24 \text{ h}}{1 \text{ día}} = 786432 \text{ bytes} = 768 \text{ Kbytes}$$

$$\text{Si poseemos una memoria de } 200 \text{ MB} = 200 \cdot 2^{10} \cdot \text{KB}$$

$$\text{nº ficheros} = \frac{\text{Memoria}}{\text{Tamaño fichero}} = \frac{200 \cdot 2^{10}}{768} = 266.6 \text{ ficheros}$$

Se generan 266 ficheros históricos completos.

PROBLEMA 4.6 El día 8 de octubre se ha ejecutado la siguiente orden en un sistema Linux:

```
% ls /var/log/sar
-rw-r--r--      1 root    root      3049952 Oct   6 23:50 sa06
-rw-r--r--      1 root    root      3049952 Oct   7 23:50 sa07
-rw-r--r--      1 root    root      2372320 Oct   8 18:40 sa08
```

¿Cada cuánto tiempo se activa el monitor sar instalado en el sistema? ¿Cuánto ocupa el registro de información almacenada cada vez que se activa el monitor?

Sabemos según la columna , que la última ejecución es a las 23:50 por lo cual el monitor se ejecuta cada 10 minutos.

El fichero completo diario ocupa 3049952 bytes, y por una activación ocupa $\frac{3049952}{6 \cdot 24} = 21180'2 \quad 21 \text{ KB}$

PROBLEMA 4.7 Indique el resultado que produce la ejecución de las siguientes órdenes sobre un sistema Linux con el monitor sar instalado:

1. sar
2. sar -A
3. sar -u 1 30
4. sar -uB -f /var/log/sa/08
5. sar -d -s 12:30:00 -e 18:15:00 -f /var/log/sa/08
6. sadc
7. sadc 2 4
8. sadc 2 4 fichero

1. Muestra la información del fichero histórico de hoy.
2. Muestra toda la información del fichero histórico de hoy.
3. Realiza una monitorización del procesador cada 1 segundo 30 veces.
4. Muestra el uso de CPU y la memoria de paginación del fichero histórico sa08.
5. Muestra las operaciones de disco (t/s) desde las 12:30 a 18:15 del fichero sa08.
6. Realiza una monitorización del sistema y la almacena en un fichero binario.
7. Realiza una monitorización cada 2 segundos, 4 veces.
8. Igual que el 7 pero lo almacena en un fichero nombrado "fichero".

PROBLEMA 4.8 Indíquese una orden (u órdenes) que se podría emplear para monitorizar los aspectos siguientes de la actividad en un computador que trabaja con el sistema operativo Linux:

1. Capacidad de memoria física ocupada por un proceso. *top/ps*
2. Número de cambios de contexto por segundo. *vmstat/sar*
3. Carga media del sistema. *uptime/sar*
4. Número de interrupciones por segundo. *vmstat/sar*
5. Capacidad libre de la unidad de disco magnético. *df*
6. Usuarios conectados a la máquina. *who*
7. Utilización del procesador en modo usuario. *top/vmstat/sar*
8. Tiempo que lleva ejecutándose un proceso. *top/ps*
9. Tiempo que tarda un proceso en ejecutarse. *time*

PROBLEMA 4.9 Después de instrumentar un programa con la herramienta gprof el resultado obtenido ha sido el siguiente:

Flat profile:

Each sample counts as 0.01 seconds.

% time	cumulative seconds	self seconds	calls	self s/call	total s/call	name
59.36	27.72	27.72	3	9.24	14.39	reduce
33.08	43.17	15.45	6	2.57	2.57	invierte
7.56	46.70	3.53	2	1.76	1.76	calcula

El grafo de dependencias muestra que `invierte()` es llamado desde el procedimiento `reduce()`.

1. ¿Cuánto tarda en ejecutarse el código propio del procedimiento `reduce()`?
2. ¿Cuál es el procedimiento más lento del programa? ¿Y el más rápido?
3. Si el procedimiento más lento de todos se sustituye por otro tres veces más rápido, ¿cuánto tiempo tardará en ejecutarse el programa?
4. Si el procedimiento `invierte()` se sustituye por una nueva versión cuatro veces más rápida, ¿qué mejora se obtendrá en el tiempo de ejecución?
5. Calcule cuál es la aceleración máxima que se podría conseguir en el tiempo de ejecución mediante la optimización del código del procedimiento `invierte()`.

1. Columna *total s/call* del *reduce* = 14'39 s

2. El más lento es: *Reduce* $t = 14'39$
El más rápido es: *Calcula* $t = 1'76$

$$3. \frac{9'24}{3} = 3'08$$

Sabemos que se ejecuta 3 veces ese procedimiento
+ *total propio de reduce* es 9'24

Sumamos la *tiempo propios de todos los procedimientos*.

$$9'24 + 15'45 + 3'53 = 28'22 \text{ s}$$

4. Si se mejora 4 veces el tiempo de *invierte*

$$t_{\text{total}} = 46'7 \text{ s}$$

$$t_{\text{invierte}} = 15'45 \text{ s}$$

$$f = \frac{15'45}{46'7} = 0'3308$$

$$t_m = (1 - f) t_0 + \frac{f}{k} t_0 = 0'6692 \cdot 46'7 + \frac{0'3308 \cdot 46'7}{4}$$

$$t_m = 35'11373$$

$$G = \frac{46'7}{35'11373} = 1'32996$$

5.

$$G_{\max} = \lim_{k \rightarrow \infty} \frac{1}{(1 - f_{\text{inverte}}) + \frac{f_{\text{inverte}}}{k}} = \frac{1}{1 - f_{\text{inverte}}}$$

Si se mejora
inverte

$$G_{\max} = \frac{1}{1 - 0'3308} = 1'4943$$

PROBLEMA 4.10 Considere la siguiente secuencia de órdenes en un sistema informático donde está instalado el monitor sar:

```
% sdc resultado
% ls -l resultado
-rw-r--r-- 1 usuario grupo 712 2006-01-16 10:55 resultado
```

- Indique qué contiene el fichero resultado y cómo se codifica esta información.
- ¿Qué orden habría que emplear para visualizar en formato ASCII toda la información capturada por el monitor de actividad en la activación anterior?
- Si el monitor sar está instalado en la máquina para ejecutarse cada tres minutos y disponemos de 50 MB de espacio en el disco duro para almacenar la información de actividad, ¿cuántos ficheros históricos diarios podremos mantener en esta instalación?

2. El fichero contiene la monitorización realizada por el monitor sdc y la información se encuentra codificada binariamente.

3. Para visualizar la información deberemos usar sar.
sar -A -f resultado.

4. 1 activación ocupa 712 bytes
1 activación cada 3 minutos

$$\frac{712 \text{ bytes}}{3 \text{ min}} \cdot \frac{60 \text{ min}}{1 \text{ h}} \cdot \frac{24 \text{ h}}{1 \text{ día}} = 341760 \text{ bytes}$$

$$= 333'75 \text{ KB}$$

tenemos una memoria de 50 MB = $50 \cdot 2^{10}$ KB

$$n^{\circ} \text{ de ficheros} = \frac{50 \cdot 2^{10}}{333'75} = 153'4082 \text{ ficheros.}$$

Podemos mantener 153 ficheros completos.

PROBLEMA 4.11 La monitorización de un programa de dibujo en tres dimensiones mediante la herramienta gprof ha proporcionado la siguiente información (por errores en la transmisión hay valores que no están disponibles):

Flat profile:

% time	cumulative seconds	self seconds	calls	self s/call	total s/call	name
xxxxx	xxxxx	15.47	3	5.16	5.16	colorea
xxxxx	xxxxx	1.89	5	0.38	0.38	interpola
xxxxx	xxxxx	1.76	1	1.76	3.65	traza
xxxxx	xxxxx	0.46				main

Call graph:

index	% time	self	children	called	name	
[1]	100.0	0.46	19.12		main	[1]
		15.47	0.00	3/3	colorea	[2]
		1.76	1.89	1/1	traza	[3]
		15.47	0.00	3/3	main	[1]
[2]	79.0	15.47	0.00	3	colorea	[2]
		1.76	1.89	1/1	main	[1]
[3]	18.6	1.76	1.89	1	traza	[3]
		1.89	0.00	5/5	interpola	[4]
		1.89	0.00	5/5	traza	[3]
[4]	9.7	1.89	0.00	5	interpola	[4]

1. ¿En cuánto tiempo se ejecuta el programa de dibujo?
2. Indique cuánto tiempo tarda en ejecutarse el código propio de main().
3. Establezca la relación de llamadas entre los procedimientos del programa así como el número de veces que se ejecuta cada uno de ellos.
4. Calcule el nuevo tiempo de ejecución del programa si se elimina el código propio de main() y se reduce a la mitad el tiempo de ejecución del código propio del procedimiento traza().
5. Proponga y justifique numéricamente una acción sobre el programa original que no afecte el procedimiento colorear() (ni su código ni el número de veces que es ejecutado) con el fin de conseguir que el programa se ejecute en 10 segundos.

1. Sumamos la columna self seconds

$$15'47 + 1'89 + 1'76 + 0'46 = 19'58 \text{ s}$$

2. El propio código main tarda 0'46 s

3. Como podemos ver según el call graph:

El código main llama 3 veces a colorear y una vez a traza.

A su vez traza llama 5 veces a interpola.

WUOLAH

4. Si $t_{main} = 0$

$$t_{traza} = \frac{t_{traz}}{2} = \frac{1'76}{2} = 0'88s$$

$$t_{colorea} = 5'16$$

$$t_{total} = 5'16 \cdot 3 + 0'88 + 1'89 = 18'25s$$

5. No se puede debido a que el propio procedimiento colorear provoca que el programa tarde al menos 15'47s como mínimo y despreciar los tiempos de los demás procedimientos.

PROBLEMA 4.12 Un informático desea evaluar el rendimiento de un computador por medio del benchmark SPEC CPU2006. Una vez compilados todos los programas del paquete y lanzado su ejecución monitoriza el sistema con la orden `vmstat 1 5`. El resultado de las medidas de este monitor es el siguiente:

procs		-----memory-----				---swap--		-----io---		---system		-- ----cpu----			
r	b	swpd	free	buff	cache	si	so	bi	bo	in	cs	us	sy	id	wa
0	0	8	14916	92292	833828	0	0	0	3	0	7	3	1	96	0
1	0	8	14916	92292	833828	0	0	0	0	1022	40	100	0	0	0
3	0	8	14916	92292	833828	2	1	16	3	1016	34	99	1	0	0
1	0	8	14916	92292	833828	0	4	0	8	1035	36	98	2	0	0
2	0	8	14916	92292	833828	1	5	4	28	1035	36	99	1	0	0

Indique si, a la vista de los datos anteriores, los resultados obtenidos en la prueba evaluación serán correctos o no. Justifique la respuesta.

PROBLEMA 4.13 El resultado de la monitorización de una aplicación informática dedicada al análisis de modelos atmosféricos se muestra a continuación (nótese que hay información no disponible):

Flat profile:

% time	cumulative seconds	self seconds	calls	self s/call	total s/call	name
xxxxx	xxxxx	30.16	52	0.58	0.58	nimbo
xxxxx	xxxxx	5.13	2	2.56	2.56	borrasca
xxxxx	xxxxx	3.51	2	1.75	1.75	lluvia
xxxxx	xxxxx	1.76	1	1.76	34.17	nube

1. Indique cuánto tiempo tarda en ejecutarse el programa.
2. Determine el porcentaje del tiempo de ejecución que consume el procedimiento lluvia().
3. ¿Cuál es el procedimiento más lento de todo el programa?
4. ¿Cuánto tiempo tarda en ejecutarse el código propio de borrasca()?
5. Calcule el nuevo tiempo de ejecución del programa si el procedimiento nimbo() se rediseña y mejora 3 veces.
6. Proponga y justifique numéricamente alguna manera de reducir el tiempo de ejecución del programa original hasta los 20 segundos.

1. El programa tarda $30'16 + 5'13 + 3'51 + 1'76 = 40'56 \text{ s}$

2. $T_{\text{total}} = 40'56 \text{ s}$

+ lluvia = $1'75 \rightarrow 2 \text{ veces}$

$$f = \frac{3'51}{40'56} = 0'0865$$

3. Para visualizar el procedimiento más lento debemos comparar la columna (total s/call) y veremos que nube es el más lento
 $t = 34'17 \text{ s}$

4. tarda $2'56 \text{ s}$

5. Si mejoramos el procedimiento nimbo $k=3$

$$f_{\text{nimbo}} = \frac{30'16}{40'56} = 0'74359$$

WUOLAH

$$t_m = \left((1 - \beta) + \frac{\beta}{k} \right) t_0 = \left(0'2564 + \frac{0'7436}{3} \right) \cdot 40'56$$

$$t_m \sim 20'4530 \text{ s}$$

6. Como podemos ver mejorando número 3 veces hemos obtenido un tiempo de mejora próximo a los 20s

Si mejoramos 4 veces,

$$t_m = \left((1 - \beta) + \frac{\beta}{k} \right) t_0 = \left(0'2564 + \frac{0'7436}{4} \right) \cdot 40'56$$

$$t_m \sim 17'9397 \text{ s}$$

De esta forma conseguimos fácilmente el objetivo.

PROBLEMA 4.14 Después de conectarse a un sistema informático, un usuario ejecuta las dos órdenes siguientes con el resultado que se muestra:

```
% uptime
 9:50am up 173 days, 23:02, 1 user, load average: 0.00, 0.00, 0.00

% time simulador
real 8m0.70s
user 3m5.20s
sys  0m4.01s
```

1. ¿En qué condición de carga se encuentra el computador (baja, media o alta) en el momento de conexión del usuario?
2. ¿Cuál es el tiempo (en segundos) de ejecución del programa simulador?
3. ¿Encuentra alguna incoherencia en los resultados anteriores? Justifique la respuesta con argumentos sólidos.

1. Como sabemos el comando `uptime` nos muestra en sus 3 últimas columnas el n° de procesos activos en los últimos 1, 5, 15 minutos.

Como podemos observar todas las columnas están a 0 eso significaría que el sistema no estaba siendo utilizado, encontrándose con una carga baja (nula).

2. El tiempo de ejecución sería de 8 minutos y 0.7s pero podemos ver como el propio programa en realidad tarda 3 minutos y 9.21s.

3. Sí, ambos resultados se contradicen ya que el comando `uptime` nos dice que el sistema no posee ninguna carga; pero `time` nos indica que el programa ha permanecido más tiempo del necesario, esto debido a una alta carga.